

LSI POO 2026 - Trabajo Práctico N°4

Alumno/s: Caceres Lourdes Gabriela – Chavez Tobías Acdel

DNI: 45.940.137 – 47.616.254

Profesor: Mónica Ringa – Daniela Demchum

Grupo Laboratorio: 1

TP: 4

Fecha de entrega: 20/09/2026

El objetivo de este trabajo fue aprender a crear, recorrer y trabajar con colecciones de objetos, recayendo todo el enfoque sobre estas mismas y como agregar, eliminar y acceder a cada elemento de una colección dinámica.

El desarrollo del trabajo fue realizado utilizando propiedades del paradigma y conceptos con los que ya veníamos trabajando como lo son el doble encapsulamiento, reutilización, colaboraciones entre objetos y polimorfismo. Para trabajar utilizamos estructuras ArrayList y HashMap para realizar colecciones dinámicas, aprendimos a trabajar con sus propios métodos y analizamos las cardinalidades de cada situación.

Reutilización

La reutilización estuvo muy presente durante el desarrollo de este trabajo, todos los ejercicios requerían reutilizar clases de trabajos anteriores para la creación de colecciones dinámicas, presentando entonces la ventaja de no tener que modificar una clase existente ni definir una nueva ya que las clases pueden volver a utilizarse por su naturaleza versátil, reduciendo además el tiempo de desarrollo y formando una estructura más eficiente.

Colecciones

Las colecciones son clases del lenguaje que nos permiten agrupar objetos y manejarlos como una sola entidad, permite de una manera más sencilla buscar, agregar y eliminar elementos que la forma en que lo harían los arrays convencionales estáticos. Trabajamos con ArrayList que simula el manejo de una lista como ya lo hacíamos en el paradigma funcional, siendo un grupo de elementos donde se ingresa, egresa y buscan elementos independientemente de su posición. Por otro lado, las estructuras de tipo HashMap funciona igual que ArrayList pero se define una clave hash mediante la cuál se realizan las operaciones básicas de la colección.

Las colecciones pueden ser estáticas cuando su tamaño es definido previamente e inmóvil, por ejemplo con un vector como el array que utilizamos para el primer ejercicio con el array de 6 puntos. No pueden haber más de 6 puntos, el array es de un único tipo y se puede recorrer con una estructura for simple.

```

System.out.println("***INGRESO DE PUNTOS***");
for(int i=0; i<6; i++){
    System.out.println("\n***PUNTO " + (i+1) + "***");
    System.out.println("Ingrese el valor x del punto.");
    int x = teclado.nextInt();
    System.out.println("Ingrese el valor y del punto.");
    int y = teclado.nextInt();
    puntos[i] = new Punto(x, y);
}

System.out.println("\n***COORDENADAS DE LOS 6 PUNTOS***");
for(int i=0; i<6; i++){
    System.out.println(puntos[i].coordenadas());
}

System.out.println("\n***DISTANCIA ENTRE PUNTOS CONSECUTIVOS***");
for(int i=0; i<5; i++){
    System.out.println("Distancia entre el punto " + (i+1) + " y el punto " + (i+2) + ": "
        + puntos[i].distanciaA(puntos[i+1]));
}

```

Las colecciones también son dinámicas, esto ocurre cuando su tamaño es cambiante mientras el programa está en ejecución, esto lo utilizamos en el resto de ejercicios con las listas y su manejo de búsqueda, agregar y eliminar elementos. Estas colecciones a su vez pueden dividirse entre genéricas y no genéricas, en el paradigma presenciamos esto cuando la colección se define ya con un tipo y no es solo un grupo de objetos. Implementamos un ejemplo de no genérico en el segundo ejercicio.

```

10 public class Curso
11 {
12     //ATRIBUTOS
13     private String nombre;
14     private HashMap<Integer, Alumno> alumnos;
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

10 public class Pedido
11 {
12     // Atributos
13     private Calendar fecha;
14     private Cliente cliente;
15     private ArrayList productos;
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Homogeneidad y heterogeneidad

Estos conceptos parten de lo ya explicado sobre las colecciones genéricas y no genéricas, siendo una colección homogénea una donde todos sus elementos son del mismo tipo, es decir, son trabajadas de forma genérica, algo así como `ArrayList<Punto>` siendo un array que solo maneja instancias de objetos de clase `Punto`. Por el contrario, las heterogéneas son más flexibles al englobar objetos sin necesidad de ser de un tipo específico, siendo conveniente para situaciones donde se tengan objetos que comparten algún comportamiento pero su identidad e implementación es distinta.